

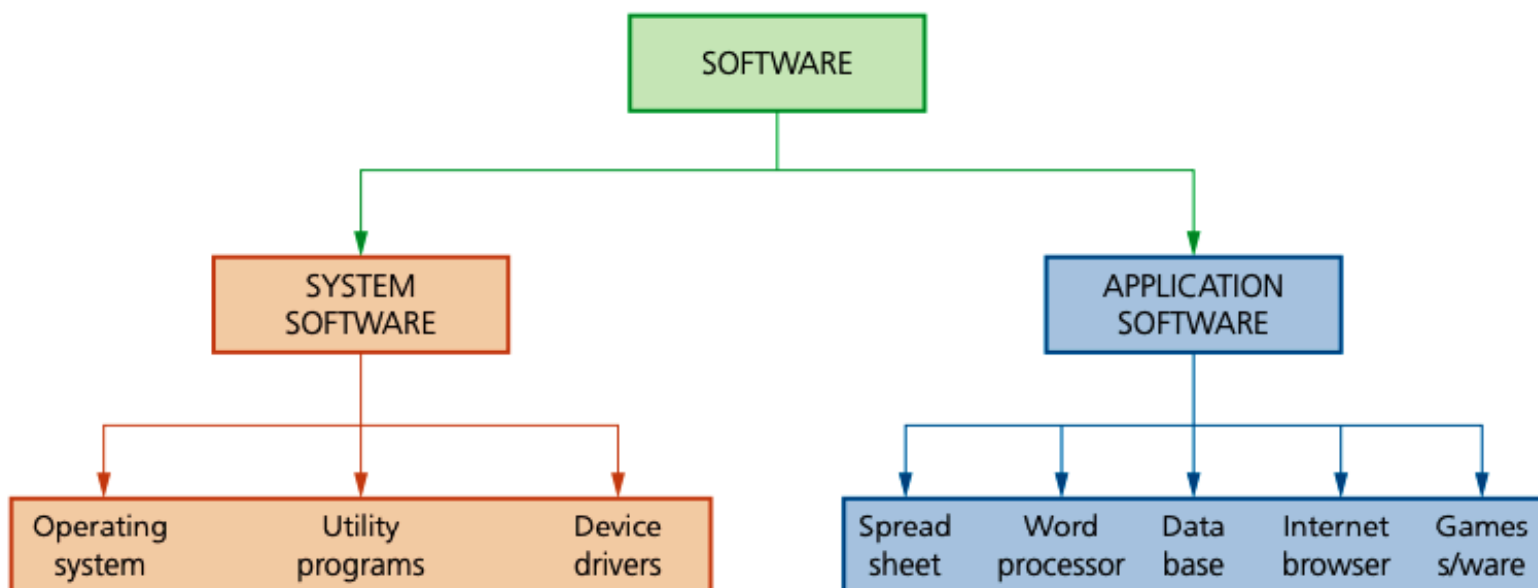
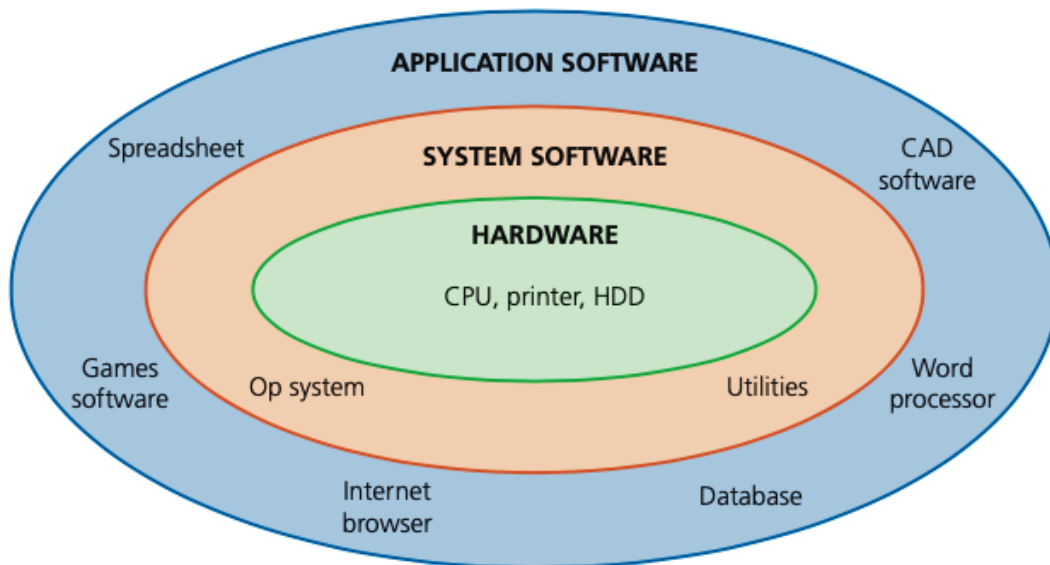
Types of software and interrupts

Software

Zain Merchant

System software and application software

All computers begin life as a group of connected hardware items. Without software, the hardware items would be useless. This section considers the link between hardware and software.



General features of system software

- set of programs to control and manage the operation of computer hardware
- provides a platform on which other software can run
- required to allow hardware and software to run without problems
- provides a human computer interface (HCI)
- controls the allocation and usage of hardware resources.

General features of application software

- used to perform various applications (apps) on a computer
- allows a user to perform specific tasks using the computer's resources
- may be a single program (for example, NotePad) or a suite of programs (for example, Microsoft Office)
- user can execute the software as and when they require.

WORD PROCESSOR:

Word processing software is used to manipulate a text document, such as an essay or a report. Text is entered using a keyboard and the software provides tools for copying, deleting and various types of formatting. Some of the functions of word processing software include:

- creating, editing, saving and manipulating text
- copy and paste functions
- spell checkers and thesaurus
- import photos/images into a structured page format
- translation into a foreign language.

SPREADSHEET:

Spreadsheet software is used to organise and manipulate numerical data (in the form of integer, real, date, and so on). Numbers are organised on a grid of lettered columns and numbered rows. The grid itself is made up of cells, and each cell is identified using a unique combination of columns and rows, for example, B6. Some of the functions of spreadsheets include:

- use of formulas to carry out calculations
- ability to produce graphs
- ability to do modelling and "what if" calculations.

DATABASE:

Database software is used to organise, manipulate and analyse data. A typical database is made up of one or more tables. Tables consist of rows and columns. Each row is called a 'record' and each column is called a 'field.' This provides the basic structure for the organisation of the data within the database. Some of the functions include:

- ability to carry out queries on database data and produce a report
- add, delete and modify data in a table.

CONTROL AND MEASURING SOFTWARE:

Control and measuring software is designed to allow a computer or microprocessor to interface with sensors so that it is possible to:

- measure physical quantities in the real world (such as temperatures)
- to control applications (such as a chemical process) by comparing sensor data with stored data and sending out signals to alter process parameters (e.g. open a valve to add acid and change the pH).

APPLICATION SOFTWARE:
these are programs that allow the user to do specific tasks.

Examples include:

APPS:

Apps is short for applications – a type of software. They normally refer to software which runs on mobile phones or tablets. They are normally downloaded from an "App Store" and range from games to sophisticated software such as phone banking. Common examples of apps include:

- video and music streaming
- GPS (global positioning systems – help you find your way to a chosen location)
- camera facility (taking photos and storing/manipulating the images taken).

PHOTO EDITING SOFTWARE:

Photo editing software allows a user to manipulate digital photographs stored on a computer; for example, change brightness, change contrast, alter colour saturation or remove "red eye". They also allow for very complex manipulation of photos (e.g. change the features of a face, combine photos, alter the images to give interesting effects and so on). They allow a photographer to remove unwanted items and generally "touch up" a photo to make it as perfect as possible.

VIDEO EDITING SOFTWARE:

Video editing software is the ability to manipulate videos to produce a new video. It enables the addition of titles, colour correction and altering/adding sound to the original video. Essentially it includes:

- rearranging, adding and/or removing sections of video clips and/or audio clips
- applying colour correction, filters and other video enhancements
- creating transitions between clips in the video footage.

GRAPHICS MANIPULATION SOFTWARE:

Graphics manipulation software allows bitmap and vector images to be changed. Bitmap images are made up of pixels which contain information about image brightness and colour. Bitmap graphics editors can change the pixels to produce a different image. Vector graphic editors operate in a different way and don't use pixels – instead they manipulate lines, curves and text to alter the stored image as required. Both types of editing software might be chosen depending on the format of the original image.

COMPILERS:

A compiler is a computer program that translates a program written in a high-level language (HLL) into machine code (code which is understood by the computer) so that it can be directly used by a computer to perform a required task. The original program is called the *source code* and the code after compilation is called the *object code*. Once a program is compiled, the machine code can be used again and again to perform the same task without re-compilation. Examples of high-level languages include: Java, Python, Visual Basic, Fortran, C++ and Algol.

LINKERS:

A linker (or link editor) is a computer program that takes one or more object file produced by a compiler and combines them into a single program which can be run on a computer. For example, many programming languages allow programmers to write different pieces of code, called modules, separately. This simplifies the programming task since it allows the program to be broken up into small, more manageable sub-tasks. However, at some point, it will be necessary to put all the modules together to form the final program. This is the job of the linker.

DEVICE DRIVERS:

A device driver is the name given to software that enables one or more hardware devices to communicate with the computer's operating system. Without drivers, a hardware device (for example, a computer printer) would be unable to work with the computer. All hardware devices connected to a computer have associated drivers. As soon as a device is plugged into the USB port of a computer, the operating system looks for the appropriate driver. An error message will be produced if it can't be found. Examples of drivers include: printers, memory sticks, mouse, CD drivers, and so on.

OPERATING SYSTEMS (O/S):

The operating system (OS) is essentially software running in the background of a computer system. It manages many of the basic functions. Without the OS, most computers would be very user-unfriendly and the majority of users would find it almost impossible to work with computers on a day-to-day basis.

For example, operating systems allow:

- input/output operations
- users to communicate with the computer (e.g. *Windows*)
- error handling to take place
- the loading and running of programs to occur
- managing of security (e.g. user accounts, log on passwords).

SYSTEM SOFTWARE:

these are programs that allow the hardware to run properly and allow the user to communicate with the computer

Examples include:

UTILITIES:

Utility programs are software that are designed to carry out specific tasks on a computer. Essentially, they are programs that help to manage, maintain and control computer resources. Examples include:

- anti-virus (virus checkers)
- anti-spyware
- back-up of files
- disk repair and analysis
- file management and compression
- security
- screensavers
- disk defragmenter/ defragmentation software.

Utility software (utilities)

Computer users are provided with a number of utility programs (often simply referred to as utilities) that are part of the system software.

Utility programs are often initiated by the user, but some, notably virus checkers, often just run in the background without the need for any user input. Utility programs offered by most computer system software include:

virus checkers

- defragmentation software
- disk contents analysis and repair
- file compression and file management
- back-up software
- security
- screensavers.

Virus checkers

Any computer (including mobile phones and tablets) can be subject to a virus attack. Operating systems offer virus checkers, but these must be kept thoroughly up to date and should run in the background to maintain their ability to guard against being infected by such malware. There are many ways to help prevent viruses (such as being careful when downloading material from the internet, not opening files or websites given in emails from unknown senders or by not using non-original software). However, virus checkers still afford the best defence against such malware.

Running anti-virus software in the background on a computer will constantly check for virus attacks. Although various types of anti-virus software work in different ways they all have the following common features:

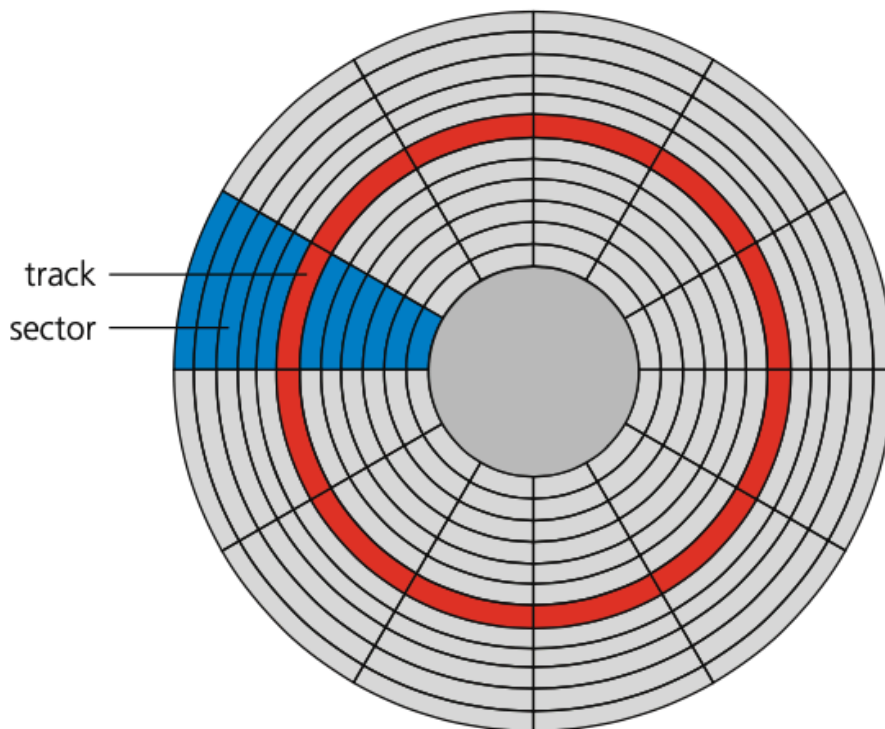
- they check software or files before they are run or loaded on a computer
- anti-virus software compares a possible virus against a database of known viruses
- they carry out heuristic checking – this is the checking of software for types of behaviour that could indicate a possible virus; this is useful if software is infected by a virus not yet on the database
- any possible files or programs which are infected are put into quarantine which:
 - allows the virus to be automatically deleted, or
 - allows the user to make the decision about deletion (it is possible that the user knows that the file or program is not infected by a virus – this is known as a false positive and is one of the drawbacks of anti-virus software)
- anti-virus software needs to be kept up to date since new viruses are constantly being discovered
- full system checks need to be carried out once a week, for example, since some viruses lie dormant and would only be picked up by this full system scan.

Defragmentation software

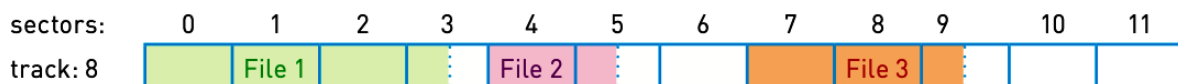
As a HDD becomes full, blocks used for files will become scattered all over the disk surface (in potentially different sectors and tracks as well as different surfaces). This will happen because files will become deleted, partially-deleted, extended and so on over time. The consequence of this is slower data access time; the HDD read-write head will now require several movements just to find and retrieve the data making up the required file.

It would obviously be advantageous if files could be stored in contiguous sectors considerably reducing HDD head movements. (Note that due to the different operation of SSDs when accessing data, this is not a problem when using solid state devices.)

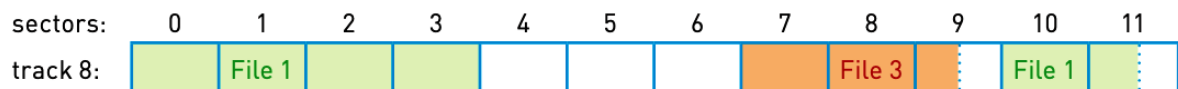
Consider the following scenario using a disk with 12 (numbered 0 to 11) sectors per surface:



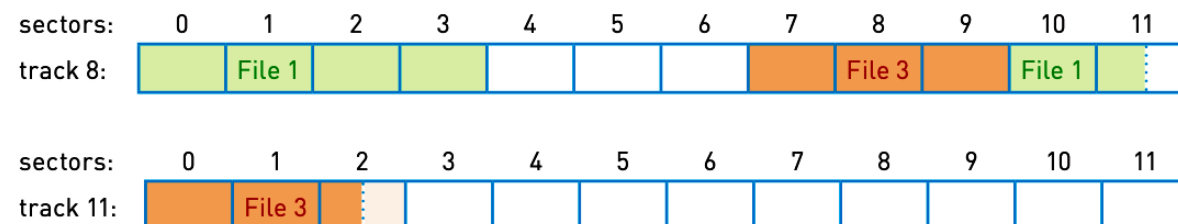
In this example we have three files (1, 2 and 3) stored on track 8 of a disk surface covering all 12 sectors:



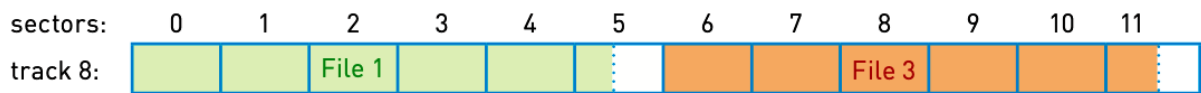
Now file 2 is deleted by the user and file 1 has data added to it; however, the file 2 sectors which become vacant are not filled up straight away by new file 1 data since this would require 'too much effort' for the HDD resources; we now get the following (file 1 is now stored in sectors 0, 1, 2, 3, 10 and 11):



File 1 has now been extended to write data in sectors 10 and 11; now suppose file 3 is extended with the equivalent of 3 blocks of data; this now requires filling up sector 9 and then finding some empty sectors to write the remainder of the data – suppose the next free sectors are on track 11:



If this continues, the files just become more and more scattered throughout the disk surfaces. It is possible for sectors 4, 5 and 6 (on track 8) to eventually become used if the disk starts to fill up and it has to use up whatever space is available. A disk defragmenter will rearrange the blocks of data to store files in contiguous sectors wherever possible. After defragmentation Track 8 would now become:



This obviously allows for much faster data access and retrieval since the HDD will now require fewer read-write head movements to access and read files 1 and 3. Track 11 would be empty after the defragmentation process.

Backup software

While it is sensible to take manual back-ups using, for example, a memory stick or portable HDD, it is also good practice to use the operating system back-up utility. This utility will:

- allow a schedule for backing up files to be made
- only carry out a back-up procedure if there have been any changes made to a file.

For total security there should be three versions of a file:

1. the current (working) version stored on the internal HDD or SSD
2. a locally backed up copy of the file (stored on a portable SSD, for example)
3. a remote back-up version stored well away from the computer (for example, using cloud storage).

The Microsoft Windows environment offers the following facilities using the back-up utility:

- restore data, files or the computer from the back-up (useful if there has been a problem and files have been lost and need to be recovered)
- create a restore point (this is basically a kind of 'time machine' where your computer can be restored to its state at this earlier point in time; this can be very useful if a very important file has been deleted and can't be recovered by any of the other utilities)
- options of where to save back-up files; this can be set up from the utility to ensure files are automatically backed up to a chosen device.

Windows uses File History, which takes snapshots of files and stores them on an external HDD at regular intervals. Over a period of time, File History builds up a vast library of past versions of files – this allows a user to choose which version of the file they want to use. File History defaults to backing up every hour and retains past versions of files for ever unless the user changes the settings.

Mac OS offers the Time Machine back-up utility. This erases the contents of a selected drive and replaces them with the contents from the back-up. To use this facility, it is necessary to have an external HDD or SSD (connected via USB port) and ensure that the Time Machine utility is installed and activated on the selected computer. Time machine will automatically:

- back-up every hour
- do daily back-ups for the past month, and
- weekly back-ups for all the previous months.

Screensavers

Screensavers are programs that supply moving and still images on the monitor screen after a period of inactivity by the computer. They were originally developed to protect older CRT (cathode ray tube) monitors which would suffer from 'phosphor burn' if the same screen image remained for any length of time. With modern LCD and OLED screens, this problem no longer exists; consequently, screensavers are now mostly just a way of customising a device. However, many screensavers are also used as part of the computer's security system. If a computer is unused for five minutes, for example, and hasn't been logged out, this will trigger the screensaver to be loaded. The computer user will then be automatically logged out and a screensaver will indicate that the computer is now locked. This gives an extra layer of security for computers used in an office environment, for example.

Some screensavers are often used to activate useful background tasks that can only go on when the computer is in an 'idle' state. For example:

- virus scans
- distributed computing applications – these allow apps to use the computer's resources only when it is idle (for example, an online gaming app).

Device drivers

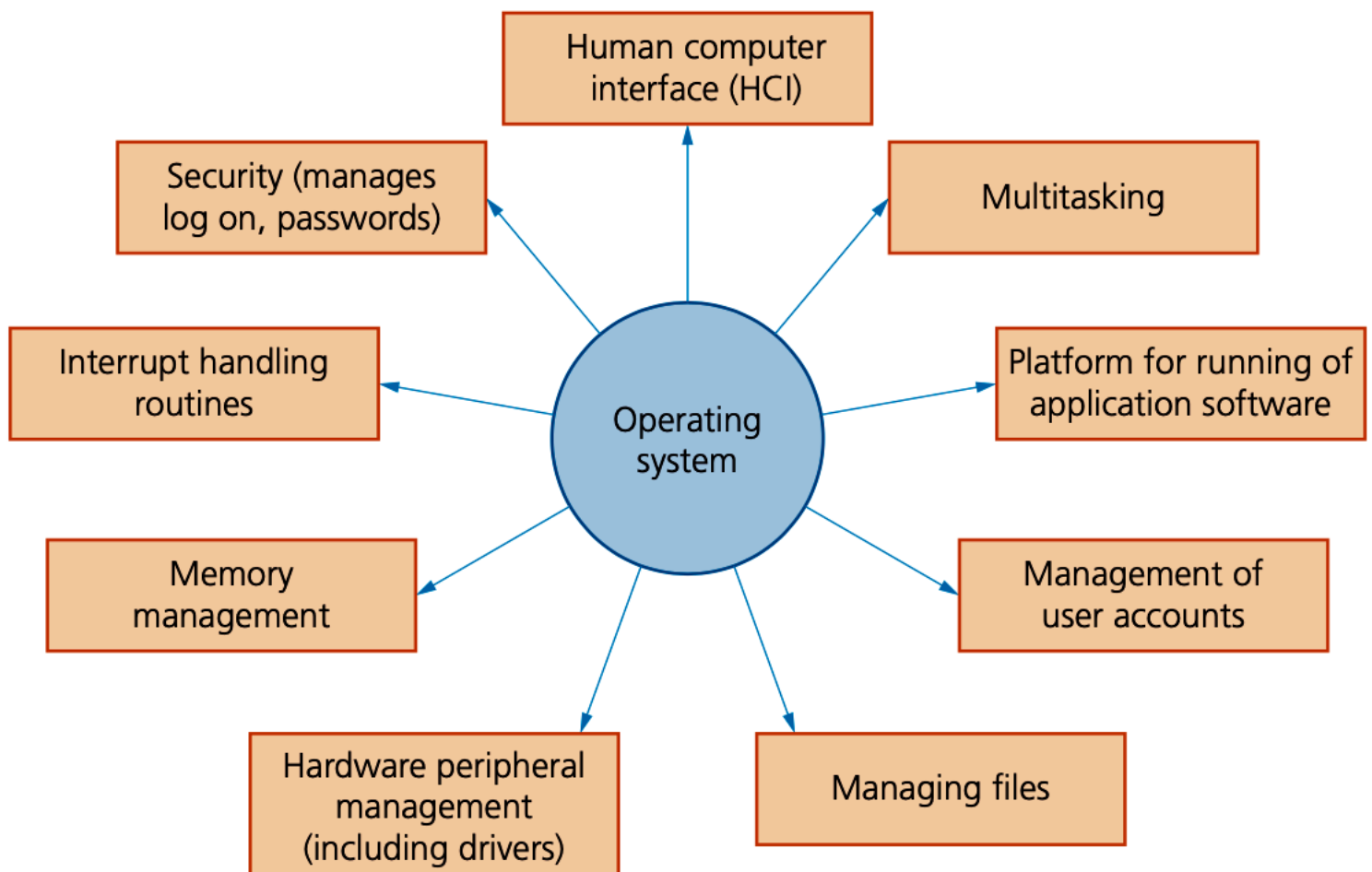
Device drivers are software that communicate with the operating system and translate data into a format understood by a hardware peripheral device. Without device drivers, a hardware device would be unable to work with a computer – a message such as ‘device not recognised’ would appear on the screen. As soon as a device is plugged into a USB port (for example, a memory stick, printer or camera), the operating system looks for the appropriate device driver.

All USB device drivers contain a collection of information about devices called descriptors; this allows the USB bus to ask a newly connected device what it is. Descriptors include vendor id (VID), product id (PID) and unique serial numbers. If a device has no serial number associated with it, the operating system will treat the device as new every time it is plugged into a USB port. Serial numbers must be unique since this could prove rather interesting if two different devices with the same serial number were plugged into a computer at the same time.

Operating systems

To enable computer systems to function correctly and allow users to communicate with computer systems, software known as an operating system needs to be installed. An operating system provides both the environment in which applications can be run and a useable interface between humans and computer. An operating system also disguises the complexity of computer software and hardware. Common examples of operating systems include: Microsoft Windows, Apple Mac OS, Google Android and Apple IOS (the latter two being used primarily on tablets and smartphones).

Most computers store the operating system on a hard disk drive (HDD) or solid state drive (SSD) since they tend to be very large programs. Mobile phones and tablets store the operating system on a solid state device since they are too small to accommodate an HDD.



Human computer interface (HCI)

The human computer interface (HCI) is in the form of a Command Line Interface (CLI) or a Graphical User Interface (GUI).

A CLI requires a user to type in instructions in order to choose options from menus, open software, etc. There are often a number of commands that need to be typed in, for example, to save or load a file. The user has to therefore learn a number of commands just to carry out basic operations. It is also slow having to key in these commands every time an operation has to be carried out. However, the advantage of CLI is that the user is in direct communication with the computer and is not restricted to a number of pre-determined options.

A GUI allows the user to interact with a computer (or MP3 player, gaming device, mobile phone, etc.) using pictures or symbols (icons) rather than having to type in a number of commands. Figure 4.8 shows a mobile screen with a number of GUI icons.

Simply selecting any of the icons from the screen would automatically load the application into the phone ready to be used. There is no need to type in anything.

GUIs use various technologies and devices to provide the user interface. One of the most common is WIMP (windows icons menu and pointing device), which was developed for use on personal computers (PC). Here a mouse is used to control a cursor and icons are selected to open/run windows. Each window contains an application and modern computer systems allow several windows to be open at the same time. An example is shown below (here a number of icons can be seen on the left-hand side and on the bottom):

A windows manager looks after the interaction between windows, the applications, the pointing devices and the cursor's position.

More recently, devices such as mobile phones and tablets increasingly use touch screens and use post-WIMP interactions. With this system, fingers are in contact with the screen allowing actions such as pinching and rotating, which would be difficult to do using a single pointer and a device such as a mouse.

Interface	Advantages	Disadvantages
command line interface (CLI)	- the user is in direct communication with the computer - the user is not restricted to a number of pre-determined options - it is possible to alter computer configuration settings - uses a small amount of computer memory	- the user needs to learn a number of commands to carry out basic operations - all commands need to be typed in which takes time and can be error-prone - each command must be typed in using the correct format, spelling, and so on
graphical user interface (GUI)	- the user doesn't need to learn any commands - it is more user-friendly; icons are used to represent applications - a pointing device (such as a mouse) is used to click on an icon to launch the application – this is simpler than typing in commands or a touch screen can be used where applications are chosen by simply touching the icon on the screen	- this type of interface uses up considerably more computer memory than a CLI interface - the user is limited to the icons provided on the screen - needs an operating system, such as Windows, to operate, which uses up considerable memory

Who would use each type of interface?

- CLI: a programmer, analyst or technician; basically somebody who needs to have a direct communication with a computer to develop new software, locate errors and remove them, initiate memory dumps (contents of the computer memory at some moment in time), and so on
- GUI: the end-user who doesn't have or doesn't need to have any great knowledge of how the computer works; a person who uses the computer to run software or play games or stores/manipulates photographs, for example.

Memory management

Memory management carries out the following functions:

- manages the primary storage (RAM) and allows data to be moved between RAM and HDD/SSD during the execution of programs
- keeps track of all the memory locations
- carries out memory protection to ensure that two competing applications cannot use the same memory locations at the same time. If this wasn't done the following might happen:
 - data would probably be lost
 - applications could produce incorrect results (based on the wrong data being in memory locations)
 - potential security issues (if data is placed in the wrong location, it might make it accessible to other software, which would be a major security issue)
 - in extreme cases, the computer could crash.

Security management

Security management is another part of a typical operating system; the function of security management is to ensure the integrity, confidentiality and availability of data. This can be achieved as follows:

- by carrying out operating system updates as and when they become available
- ensuring that anti virus software (and other security software) is always up to date, preserving the integrity, security and privacy of data
- by communicating with, for example, a firewall to check all traffic to and from the computer
- by making use of privileges to prevent users entering 'private areas' on a computer that permits multi-user activity (this is done by setting up user accounts and making use of passwords and user IDs); this helps to ensure the privacy of data
- by maintaining access rights for all users
- by offering the ability for the recovery of data (and system restore) when it has been lost or corrupted
- by helping to prevent illegal intrusion into the computer system (also ensuring the privacy of data).

Hardware peripheral management

Hardware management involves all input and output peripheral devices. Hardware management:

- communicates with all input and output devices using device drivers
- uses a device driver to take data from a file (defined by the operating system) and translates it into a format that the input/output device can understand
- ensures each hardware resource has a priority so that they can be used and released as required
- manages input/output devices by controlling queues and buffers; consider the role of the printer management when printing out a document:
 - first of all, the printer driver is located and loaded into memory
 - then the data is sent to a printer buffer ready for printing
 - if the printer is busy (or the printing job has a low priority) then the data is sent to a printer queue before it can be sent to the printer buffer
 - it will send various control commands to the printer throughout the printing process
 - it receives and handles error messages and interrupts from the printer.

File management

The main tasks of file management include:

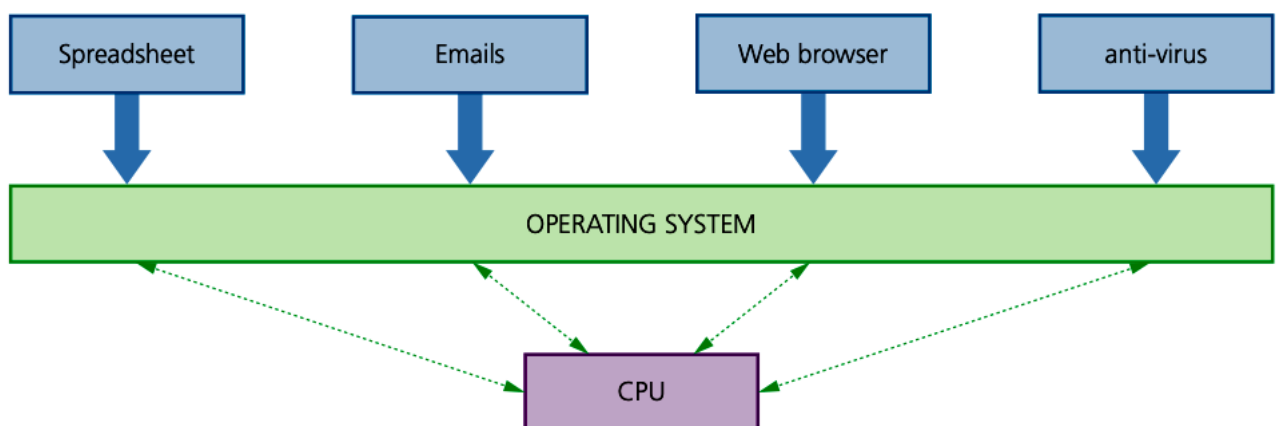
- file naming conventions which can be used i.e. filename.docx (where the extension can be .bat, .htm, .dbf, .txt, .xls, etc.)
- performing specific tasks (for example, create, open, close, delete, rename, copy, and move)
- maintaining the directory structures
- ensuring access control mechanisms are maintained (for example, access rights to files, password protection, or making files available for editing or locking them)
- ensuring memory allocation for a file by reading it from the HDD/SSD and loading it into memory.

Multitasking

Multitasking allows computers to carry out more than one task (i.e. a process) at a time. Each of the processes will share the hardware resources under the control of the operating system software. To make sure that multitasking operates correctly (in other words, the processes don't clash with each other), the operating system needs to constantly monitor the status of each of the processes under its control:

- resources are allocated to a process for a specific time limit
- the process can be interrupted while it is running
- the process is given a priority so it can have resources according to its priority (the risk here is that a low priority process could be starved of resources).

Using multitasking management, main memory, HDD/SSD and virtual memory are better managed making the most effective use of CPU time.



Management of user accounts

Computers allow more than one user to log onto the system. It is therefore important that users' data is stored in separate parts of the memory for security reasons (also refer to security management earlier in this section). Each person logging onto the computer will be given a user account protected by a user name and password. The operating system is given the task of managing these different user accounts. This allows each user to:

- customise their screen layout and other settings
- use separate folders and files and to manage these themselves.

Very often an administrator oversees the management of these user accounts. The administrator can create accounts, delete user accounts and restrict user account activity. On large university or industrial computers, part of the operating system's tasks will be to oversee several users' accounts, since

a complex multi-user system may be in place. The operating system has to maintain accounts for several users, managing data that may range from personal data and technical research work down to the ordering of stationery. Multi-access levels permit this control to take place. For example, a clerk in the office may have access to ordering stationery but can't have access to any personal data.

Running of applications

When a computer starts up, part of the operating system needs to be loaded into RAM – this is known as booting up the computer (or a bootstrap loader). The start-up of the computer's motherboard is handled by the basic input/output system (BIOS). The BIOS tells the computer where the storage device that holds the operating system can be found; it then loads the part of the operating system that is needed and executes it.

The BIOS is often referred to as firmware. Firmware is defined as a program that provides low level control for devices.

The BIOS program is stored in a special type of ROM, called an EEPROM (Electrically Erasable Programmable ROM). EEPROM is a flash memory chip, which means its contents remain even when the computer is powered down. However, it also means the BIOS can be rewritten, updated or even deleted by a user.

However, while the BIOS is stored on an EEPROM, the BIOS settings are stored on a CMOS chip (Complementary Metal Oxide Semi-conductor). The CMOS is powered up at all times via a rechargeable battery on the motherboard. Therefore, the BIOS settings would be reset if the battery was removed or disconnected

for some reason. Once the CMOS is re-started, it will access the same BIOS program from EEPROM, but the settings will now be the default factory settings. Consequently, if a user has changed the BIOS settings (for example, the clock speed), the settings will revert to those settings made at the factory once power is restored to the CMOS.



The application software will be under the control of the operating system and will need to access system software such as the device drivers while it is running. Different parts of the operating system may need to be loaded in and out of RAM as the software runs.

Interrupts

An interrupt is a signal sent from a device or from software to the microprocessor. This will cause the microprocessor to temporarily stop what it is doing so that it can service the interrupt. Interrupts can be caused by:

- a timing signal
- an input/output process (for example, a disk drive or printer requiring more data)
- a hardware fault (for example, a paper jam in the printer)
- user interaction (for example, the user presses a key (or keys) on a keyboard, such as <CTRL><ALT><BREAK>, which causes the system to be interrupted)

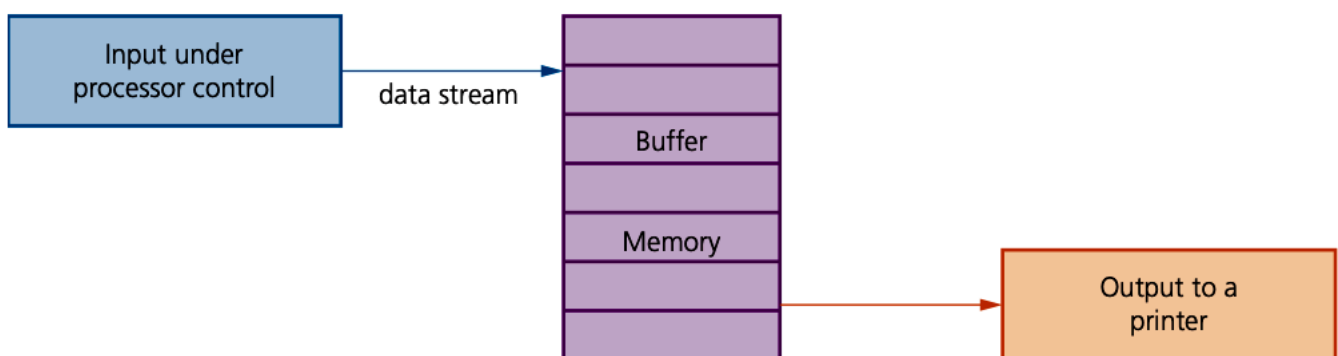
software errors that cause a problem (for example, an .exe file that cannot be found to initiate the execution of a program, two processes trying to access the same memory location, or an attempt to divide by zero).

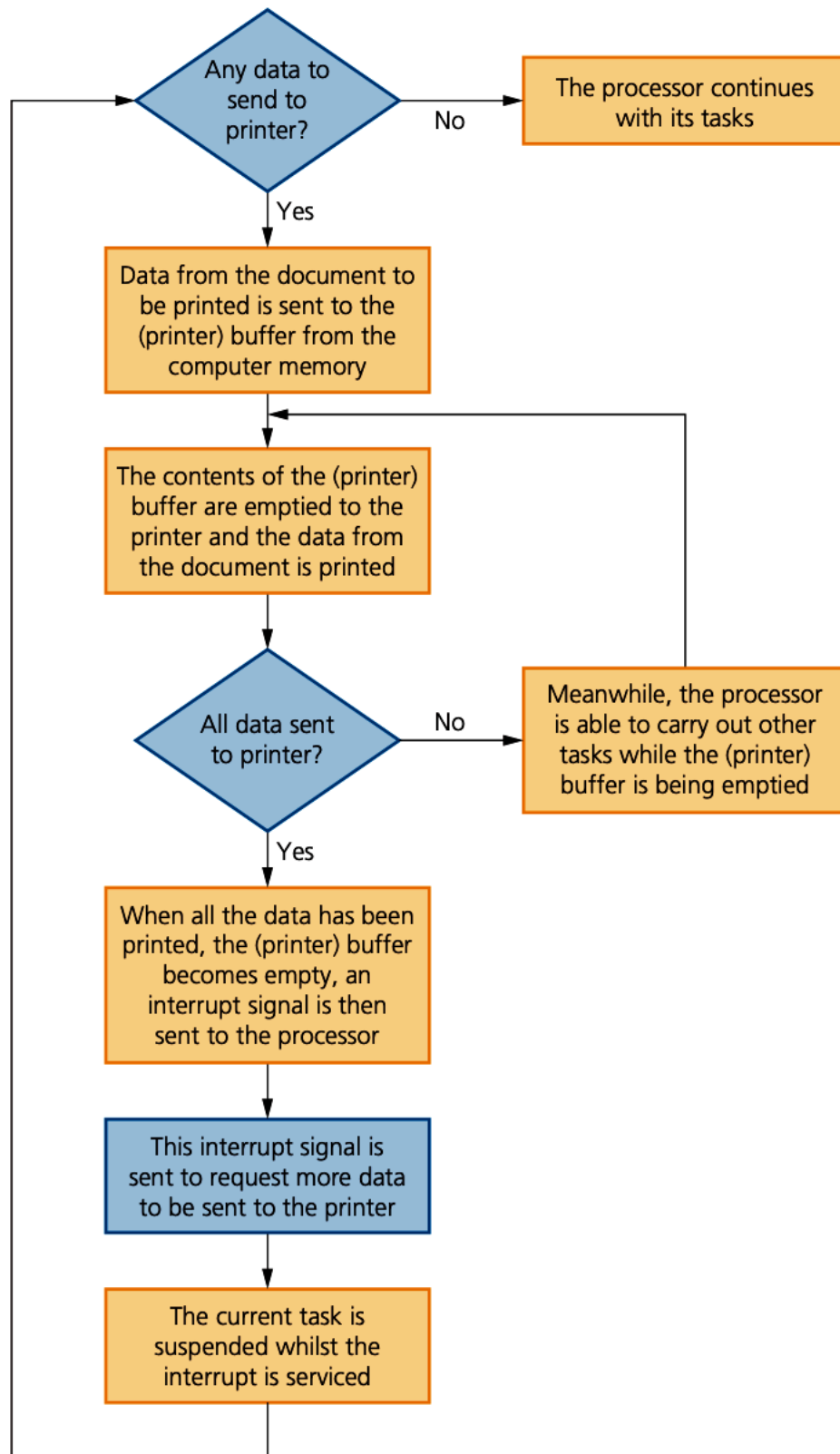
Once the interrupt signal is received, the microprocessor either carries on with what it was doing or stops to service the device or program that caused the interrupt. The computer needs to identify the interrupt type and also establish the level of interrupt priority.

Interrupts allow computers to carry out many tasks or to have several windows open at the same time. An example would be downloading a file from the internet at the same time as listening to some music from a library. Interrupts allow these two functions to co-exist and the user has the impression that both functions are being carried out simultaneously. In reality, data is being passed in and out of memory very rapidly allowing both functions to be serviced. This can all be achieved by using an area in memory known as a buffer. A buffer is a memory area that stores data temporarily. For example, buffers are used when downloading a movie from the internet to compensate for the difference between download speeds and the data requirements of the receiving device. The data transmission rate of the movie file from the web server to the buffer must be greater than the rate at which data is transmitted from buffer to media player. Without buffers, the movie would frequently 'freeze'.

Whenever an interrupt is received it needs to be serviced. The status of the current task being run first needs to be saved. The contents of the Program Counter (PC) and other registers are saved. Then the interrupt service routine (ISR) is executed by loading the start address into the Program Counter (PC). Once the interrupt has been fully serviced, the status of the interrupted task is reinstated (the contents of all the saved registers are then retrieved) and the process continues.

Buffers and interrupts are often used together to allow standard computer functions to be carried out. These functions are often taken for granted by users of modern computer systems. For example, the following diagram shows how buffers and interrupts are used when a document is sent from memory to a printer. The important thing to remember here is the time taken to print out a document is much longer than the time it takes for the microprocessor to send data to the printer. Without buffers and interrupts, the microprocessor would remain idle waiting for a document to be printed. This would be an incredible waste of microprocessor time; the buffers and interrupts allow the microprocessor to carry on with other tasks while the document is being printed thus maximising its processing power and speed.





Types of programming language, translators and integrated development environments (IDE) Software

Zain Merchant

People use many different languages to communicate with each other. In order for two people to understand each other they need to speak the same language or another person, an interpreter, is needed to translate from one language to the other language.

Programmers use many different programming languages to communicate with computers. Computers only 'understand' their own language, called machine code. A program needs to be translated into machine code before it can be 'understood' by a computer.

Programs are our way of telling a computer what to do, how to do it and when to do it. This enables a single computer to perform many different types of task. A computer can be used to stream videos, write reports, provide weather forecasts and many, many other jobs.

Here is an example of a simple task that can be performed by a computer:

A computer program is a list of instructions that enable a computer to perform a specific task. Computer programs can be written in high-level languages and low-level languages depending on the task to be performed and the computer to be used. Most programmers write programs in high-level languages.

High-level languages and low-level languages

High-level languages

High-level languages enable a programmer to focus on the problem to be solved and require no knowledge of the hardware and instruction set of the computer that will use the program. Many high-level programming languages are portable and can be used on different types of computer.

High-level languages are designed with programmers in mind; programming statements are easier to understand than those written in a low-level language. This means that programs written in a high-level language are easier to:

- read and understand as the language used is closer to English
- write in a shorter time
- debug at the development stage
- maintain once in use.

The following snippet of program to add two numbers together is a single program statement written in a typical high-level language. It shows how easy it is to understand what is happening in a high-level language program:

Sum := FirstNumber + SecondNumber

There are many different high-level programming languages in use today including C++, Delphi, Java, Pascal, Python, Visual Basic and many more. Once a programmer has learned the techniques of programming in any high-level language, these can be transferred to writing programs in other high-level languages.

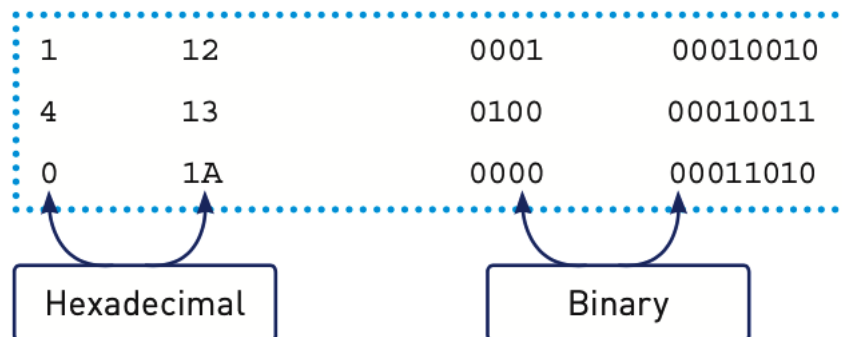
Low-level languages

Low-level languages relate to the specific architecture and hardware of a particular type of computer. Low-level languages can refer to machine code, the binary instructions that a computer understands, or assembly language that needs to be translated into machine code.

Machine code

Programmers do not usually write in machine code as it is difficult to understand, and it can be complicated to manage data manipulation and storage.

The following snippet of program to add two numbers together is written in typical machine code, shown in both hexadecimal and binary, and consists of three statements:



Language	Advantages	Disadvantages
High-level	- independent of the type of computer being used - easier to read, write and understand programs - quicker to write programs - programs are easier and quicker to debug - easier to maintain programs in use	- programs can be larger - programs can take longer to execute - programs may not be able make use of special hardware
Low-level	- can make use of special hardware - includes special machine-dependent instructions - can write code that doesn't take up much space in primary memory - can write code that performs a task very quickly	- it takes a longer time to write and debug programs - programs are more difficult to understand

Assembly languages

Software

Zain Merchant

Fewer programmers write programs in an assembly language. Those programmers who do, do so for the following reasons:

- to make use of special hardware
- to make use of special machine-dependent instructions
- to write code that doesn't take up much space in primary memory
- to write code that performs a task very quickly.

The following snippet of program to add two numbers together is written in a typical assembly language and consists of three statements:

LDA	First
ADD	Second
STO	Sum

In order to understand this program, the programmer needs to know that:

- LDA means load value of the variable (in this case, First) into the accumulator
- ADD means add value of variable (in this case, Second) to the value stored in the accumulator
- STO replace the value of the variable (in this case, Sum) by the value stored in the accumulator

Translators

Computer programs can exist in several forms.

Programs are written by humans in a form that people who are trained as computer programmers can understand. In order to be used by a computer, programs need to be translated into the binary instructions, machine code, that the computer understands. Humans find it very difficult to read binary, but computers can only perform operations written in binary.

A program must be translated into binary before a computer can use it; this is done by a utility program called a translator. There are several types of translator program in use; each one performs a different task.

Compilers

A compiler is a computer program that translates an entire program written in a high-level language (HLL) into machine code all in one go so that it can be directly used by a computer to perform a required task. Once a program is compiled the machine code can be used again and again to perform the same task without re-compilation. If errors are detected, then an error report is produced instead of a compiled program.

The high-level program statement:

```
Sum := FirstNumber + SecondNumber
```

becomes the following machine code instructions when translated:

0001	00010010
0100	00010011
0000	00011010

Interpreters

An interpreter is a computer program that reads a statement from a program written in a high-level language, translates it, performs the action specified and then does the same with the next statement and so on. If there is an error in the statement then execution ceases and an error message is output, sometimes with a suggested correction.

A program needs to be interpreted again each time it is run.

Assemblers

An assembler is a computer program that translates a program written in an assembly language into machine code so that it can be directly used by a computer to perform a required task. Once a program is assembled the machine code can be used again and again to perform the same task without re-assembly.

The assembly language program statements:

LDA	First
ADD	Second
STO	Sum

become the following machine code instructions when translated:

0001	00010010
0100	00010011
0000	00011010

Compiler	Interpreter	Assembler
Translates a high-level language program into machine code.	Executes a high-level language program one statement at a time.	Translates a low level assembly language program into machine code.
An executable file of machine code is produced.	No executable file of machine code is produced.	An executable file of machine code is produced.
One high-level language statement can be translated into several machine code instructions.	One high-level language program statement may require several machine code instructions to be executed.	One low-level language statement is usually translated into one machine code instruction.
Compiled programs are run without the compiler.	Interpreted programs cannot be run without the interpreter.	Assembled programs are used without the assembler.
A compiled program is usually distributed for general use.	An interpreter is often used when a program is being developed.	An assembled program is usually distributed for general use.

Advantages and disadvantages of compilers and interpreters Software

Zain Merchant

Translators	Advantages	Disadvantages
Interpreter	easier and quicker to debug and test programs during development easier to edit programs during development	programs cannot be run without the interpreter programs can take longer to execute
Compiler	a compiled program can be stored ready for use a compiled program can be executed without the compiler a compiled program takes up less space in memory when it is executed a compiled program is executed in a shorter time	it takes a longer time to write, test and debug programs during development

Integrated development environment (IDE)

Software

Zain Merchant

An Integrated Development Environment (IDE) is used by programmers to aid the writing and development of programs. There are many different IDEs available; some just support one programming language, others can be used for several different programming languages. You may be using PyCharm (for Python), Visual Studio (for Visual Basic) or BlueJ (for Java) as your IDE.

IDEs usually have these features:

code editors

- a translator
- a runtime environment with a debugger » error diagnostics
- auto-completion
- auto-correction
- an auto-documenter and prettyprinting.

Let's look at each of these features in turn and see how they help the development process.

Code editor

A code editor allows a program to be written and edited without the need to use a separate text editor. This speeds up the program development process, as editing can be done without changing to a different piece of software each time the program needs correcting or adding to.

Translator

Most IDEs usually provide a translator, this can be a compiler and/or an interpreter, to enable the program to be executed. The interpreter is often used for developing the program and the compiler to produce the final version of the program to be used.

A runtime environment with a debugger

A debugger is a program that runs the program under development and allows the programmer to step through the program a line at a time (single stepping) or to set a breakpoint to stop the execution of the program at a certain point in the source code. A report window then shows the contents of the variables and expressions evaluated at that point in the program. This allows the programmer to see if there are any logic errors in the program and check that the program works as intended.

Error diagnostics and auto-correction

Dynamic error checking finds possible errors as the program code is being typed, alerts the programmer at the time and provides a suggested correction. Many errors can therefore be found and corrected during program writing and editing before the program is run.

Auto-completion

Code editors can offer context-sensitive prompts with text completion for variable names and reserved words.

Auto-documenter and prettyprinting

IDEs can provide an auto-documenter to explain the function and purpose of programming code.

Most code editors colour code the words in the program and lay out the program in a meaningful way – this is called prettyprinting.